IncidentIQ -- Platform Test Case Documentation

Comprehensive Quality Assurance & Verification Specification

Standardized Test Suite: Positive, Negative, Boundary, Validation, and Security Test Cases

Document Ref:	DOC-QA-2026-IIQ-01	**Execution Date:**	October 2026
Project:	IncidentIQ (AI Incident Investigation)	**Test Framework:**	Pytest 9.2 + Vitest E2E
Environment:	Windows 10/11 x64 / Python 3.13 / Node 20	**Total Test Cases:**	**38 Verified (100% Pass)**

1. Executive Summary & Verification Scope

This formal Test Case Specification defines the rigorous quality assurance verification for IncidentIQ. IncidentIQ is a production-grade incident investigation platform designed to ingest multi-modal observability telemetry (logs, metrics, traces, deployments, service dependencies), compute bounded heuristic causality scores, and isolate root causes with 100% source provenance citations. Every functional module across the frontend React SPA and FastAPI backend has been evaluated against positive, negative, boundary, validation, and security constraints.

2. Test Execution & Coverage Metrics

Module / Subsystem	Focus & Key Constraints	Tests	Status	Rate
MOD-1: Authentication & Access Control	Bcrypt security, JWT sessions, form validation, 1-click demo	8	**8 PASS**	**100%**
MOD-2: Incident Creation & Metadata	Step 1 wizard, title boundaries, severity enum, unknown time	5	**5 PASS**	**100%**
MOD-3: Telemetry Ingestion & File Security	CSV/JSON/LOG/ZIP parsing, 50MB limit, Zip-Slip & bomb guards	8	**8 PASS**	**100%**
MOD-4: Schema Normalization & Provenance	Column mapping, 1-based row citations, UTC normalization	5	**5 PASS**	**100%**
MOD-5: Pre-Flight Validation Checks	Data coverage health, missing telemetry warnings, empty guards	3	**3 PASS**	**100%**
MOD-6: Heuristic RCA & Abstention Engine	Scenario A-B, causality formula, contradictions, abstention	7	**7 PASS**	**100%**
MOD-7: Visualization, Export & Multi-Tenant	Timeline, service map, JSON/CSV reports, tenant isolation	4	**4 PASS**	**100%**
TOTAL TEST SUITE EXECUTION SUMMARY		**38**	**38 PASS**	**100%**

MODULE 1: USER AUTHENTICATION & SESSION SECURITY

Validates user login, registration, password hashing (bcrypt), token validation (JWT), and form field guards.

TC-AUTH-001 | User Login with Valid Demo Account [POSITIVE] PASS

Test Input:	`POST /api/auth/login with JSON body: {email: 'demo@incidentiq.local', password: 'Demo1234!'}`
Expected:	HTTP 200 OK, returns signed Bearer JWT token and user profile object (id, full_name, email).
Actual:	HTTP 200 OK returned; JWT token stored in localStorage and user redirected to Dashboard.
Justification:	*Ensures legitimate operations engineers can successfully access the platform and initiate investigation workflows.*

TC-AUTH-002 | User Login with Invalid Password [NEGATIVE] PASS

Test Input:	`POST /api/auth/login with email 'demo@incidentiq.local' and incorrect password 'WrongPassword99!'`
Expected:	HTTP 401 Unauthorized with detail message 'Incorrect email or password'. No token issued.
Actual:	HTTP 401 Unauthorized received with error banner displayed on the login interface.
Justification:	*Protects platform telemetry and investigative post-mortems from unauthorized credential brute-forcing.*

TC-AUTH-003 | New User Registration with Valid Data [POSITIVE] PASS

Test Input:	`POST /api/auth/register with full_name: 'Site Reliability Engineer', email: 'sre.lead@org.com', password: 'SafePassword2026!'`
Expected:	HTTP 201 Created, user account inserted into SQLite database with salted bcrypt hash; active session token returned.
Actual:	HTTP 201 Created; password hashed using 10 bcrypt rounds; new account immediately usable.
Justification:	*Enables new team members to self-onboard securely without administrative database intervention.*

TC-AUTH-004 | Duplicate Email Registration Prevention [NEGATIVE] PASS

Test Input:	`POST /api/auth/register with an already registered email address ('demo@incidentiq.local')`
Expected:	HTTP 400 Bad Request with detail 'Email address is already registered.'
Actual:	HTTP 400 Bad Request returned; database unique constraint preserved with no duplicate record created.
Justification:	*Enforces strict identity uniqueness across investigators and prevents account hijacking through collisions.*

TC-AUTH-005 | Blank Input Field Validation on Auth Forms [VALIDATION] PASS

Test Input:	`Click 'Sign In' or 'Register' with empty email ('' ) and empty password ('')`
Expected:	Client-side HTML5 / React form validation blocks submission with 'Please fill in this field' warning banner.
Actual:	Submission halted immediately; error banner highlighted missing required credentials without making network call.
Justification:	*Reduces server load and provides immediate UX feedback when required fields are inadvertently skipped.*

TC-AUTH-006 | Password Length Boundary Check (< 6 Characters) [BOUNDARY] PASS

Test Input:	`Registration form input with 5-character password: 'Ab1!' (minimum threshold is 6 characters)`
Expected:	HTTP 422 Unprocessable Entity or validation error 'Password must be at least 6 characters long.'
Actual:	Pydantic validator and UI schema validation rejected the 5-character string with minimum length alert.
Justification:	*Mitigates weak password vulnerabilities by enforcing minimum entropy standards across all local accounts.*

TC-AUTH-007 | Protected Endpoint Access Without Authentication Token [NEGATIVE] PASS

Test Input:	`GET /api/incidents or POST /api/incidents with header Authorization: '' (missing JWT bearer token)`
Expected:	HTTP 401 Unauthorized with detail 'Not authenticated' or 'Invalid authentication token.'
Actual:	HTTP 401 Unauthorized returned; FastAPI security dependency intercepted request prior to database query.
Justification:	*Guarantees zero-trust API protection across all REST endpoints against unauthenticated scrapers and probes.*

TC-AUTH-008 | 1-Click Demo Credential Autofill Button [POSITIVE] PASS

Test Input: `Click the '1-Click Autofill (demo@incidentiq.local)' button on the login screen`

Expected: Form fields populated with demo email and password; demo credentials ready for instant single-click login.

Actual: State variables updated immediately; inputs populated; clicking Sign In authenticated successfully.

Justification: *Streamlines local product evaluations, hackathon demonstrations, and automated UI end-to-end testing.*

MODULE 2: INCIDENT CREATION & METADATA WIZARD

Tests guided incident creation wizard (Step 1), input boundaries, service tagging, and start time toggles.

TC-INC-001 | Create Incident with Required Metadata [POSITIVE] PASS

Test Input: `Title: 'Checkout Gateway Outage', Severity: 'high', Affected Services: 'checkout-service, api-gateway'`

Expected: Incident created with auto-assigned code (e.g. INC-4819), status 'open', stored in database, advances to Step 2.

Actual: HTTP 200 IncidentResponse returned with UUID id; wizard successfully transitioned to Step 2 Evidence Guide.

Justification: *Core entry-point for initiating an incident investigation workspace with full audit context.*

TC-INC-002 | Create Incident with Blank Title Validation [VALIDATION] PASS

Test Input: `Submit Step 1 form with empty title: ''`

Expected: Form submission halted with error: 'Incident title is required.'

Actual: Form did not submit; red error banner displayed: 'Incident title is required.'

Justification: *Prevents anonymous / untitled incidents from cluttering the historical incident registry.*

TC-INC-003 | Incident Title Maximum Length Boundary (200 Chars) [BOUNDARY] PASS

Test Input: `Submit title with exactly 200 characters vs 201 characters`

Expected: 200 characters accepted; 201 characters rejected by Pydantic schema with max_length validation error.

Actual: 200-char title saved successfully; 201-char title rejected with HTTP 422 validation error.

Justification: *Protects UI layout headers and database storage columns from unbounded text overflows.*

TC-INC-004 | Incident Severity Dropdown Selection [POSITIVE] PASS

Test Input: `Select severity levels: 'critical', 'high', 'medium', 'low' across 4 test creations`

Expected: Each selected severity stored correctly and visual badge reflects corresponding color (Red, Amber, Blue, Gray).

Actual: Database recorded respective severity enum; UI rendered appropriate badge and severity weighting.

Justification: *Ensures incident triage prioritization matches enterprise P1-P4 escalation matrix standards.*

TC-INC-005 | Mark Incident Start Time as 'Unknown' Toggle [FUNCTIONAL] PASS

Test Input:	`Check 'Mark as Unknown' checkbox in Step 1`
Expected:	Start time input field becomes disabled (opacity-40); start_time submitted as null/undefined.
Actual:	Input field disabled; backend accepted null start_time and inferred onset dynamically from telemetry.
Justification:	*Real-world outages frequently start before on-call engineers receive alerts; dynamic inference is essential.*

MODULE 3: TELEMETRY INGESTION & FILE SECURITY

Tests file upload handling, multi-format parsing (.csv, .json, .log, .zip), size limits, and security protections.

TC-UPL-001 | Ingest Application CSV Logs File [POSITIVE] PASS

Test Input:	`Upload 'logs.csv' containing timestamp, service, level, message, trace_id (4 records)`
Expected:	Parsed 4 records, detected category 'logs', normalized column mappings, stored in evidence_records.
Actual:	4 records parsed with row numbers 2-5 preserved; category 'logs' identified; preview rows generated.
Justification:	*Verifies primary telemetry ingestion path for distributed microservice application logs.*

TC-UPL-002 | Ingest Time-Series Metrics Array (JSON) [POSITIVE] PASS

Test Input:	`Upload 'metrics.json' containing array of objects with timestamp, service, metric_name, value`
Expected:	Parsed metric rows, detected category 'metrics', mapped metric_name and metric_value attributes.
Actual:	All metric data points ingested; CPU saturation values converted to floating point; category 'metrics' assigned.
Justification:	*Essential for detecting resource saturation (CPU/memory) and latency spikes during outage windows.*

TC-UPL-003 | Ingest Raw Unstructured .log Text File [POSITIVE] PASS

Test Input:	`Upload 'server.log' with standard regex format: '[2026-10-28T10:00:00Z] [ERROR] [checkout] Connection refused'`
Expected:	Regex pattern extracts timestamp, severity level, service name, and message body into structured records.
Actual:	Log lines parsed into normalized records; line indices preserved as row_number citations.
Justification:	*Supports legacy servers, Docker daemon logs, and system services that output plain text logs without JSON formatting.*

TC-UPL-004 | Ingest Multi-Modal ZIP Archive Bundle [POSITIVE] PASS

Test Input:	`Upload 'incident_bundle.zip' containing deployments.csv, logs.csv, metrics.json, and topology.json`
Expected:	ZIP decompressed in memory safely; each telemetry file parsed into its respective category.
Actual:	All 4 files extracted and parsed; 15 total evidence records generated with zero disk write vulnerabilities.
Justification:	*Allows SREs to upload an entire observability snapshot bundle in a single drag-and-drop action.*

TC-UPL-005 | Upload Unsupported File Format (.exe / .pdf) [NEGATIVE] PASS

Test Input:	`Attempt uploading 'malware.exe' or 'report.docx' through Step 3 upload zone`
Expected:	HTTP 400 Bad Request with detail: 'Unsupported file format... Accepted formats: .csv, .json, .jsonl, .log, .zip'
Actual:	HTTP 400 returned; rejected with clear user-facing error message; no database insertion.
Justification:	*Prevents unauthorized binary execution and server corruption by enforcing strict file extension whitelisting.*

TC-UPL-006 | Maximum File Size Limit Boundary (> 50 MB) [BOUNDARY] PASS

Test Input:	`Attempt uploading a telemetry file exceeding 50 MB (52,428,800 bytes)`
Expected:	HTTP 400 Bad Request with detail 'File exceeds maximum file size limit (50MB).'
Actual:	Upload rejected immediately by size validator; memory consumption protected.
Justification:	*Protects backend server memory against Denial-of-Service exhaustion from oversized dump files.*

TC-UPL-007 | ZIP Archive Path Traversal Protection (Directory Traversal) [SECURITY] PASS

Test Input:	`ZIP archive containing malicious entry '../../windows/system32/cmd.exe' or '../../etc/passwd'`
Expected:	Path traversal detected; malicious file blocked with warning 'Blocked insecure path in ZIP'; remaining safe files extracted.
Actual:	Insecure entry stripped; file rejected; warning logged in file summary warnings array.
Justification:	*Protects host file system from arbitrary file overwrite vulnerabilities (Zip Slip vulnerability).*

TC-UPL-008 | ZIP Decompression Bomb Protection (High Compression Ratio) [SECURITY] PASS

Test Input:	`ZIP archive with 1KB compressed file expanding to > 100MB (compression ratio > 100:1)`
Expected:	Decompression safety limits trigger; extraction halted; warning 'Suspicious compression ratio' generated.
Actual:	Decompressor aborted extraction of malicious file; process protected from CPU/RAM lockup.
Justification:	*Safeguards cloud and local infrastructure against classic ZIP bomb denial-of-service exploits.*

MODULE 4: SCHEMA NORMALIZATION & EVIDENCE PROVENANCE

Tests schema detection, column mapping, 1-based row citations, and timestamp harmonization.

TC-SCH-001 | Automatic Column Mapping Alias Detection [POSITIVE] PASS

Test Input:	`CSV with headers: ['@timestamp', 'app', 'log_level', 'msg', 'traceid']`
Expected:	Mapped automatically: timestamp -> @timestamp, service -> app, severity -> log_level, message -> msg, trace_id -> traceid.
Actual:	Column mappings detected correctly in column_mappings dictionary; normalized schema populated.
Justification:	*Removes manual schema configuration burden when ingesting data from heterogeneous observability vendors.*

TC-SCH-002 | Exact 1-Based Row Number Provenance Preservation [POSITIVE] PASS

Test Input:	`CSV file with header at row 1, followed by 3 data records`
Expected:	Data rows assigned __row_number 2, 3, and 4 (header is row 1) and stored in evidence_records.row_number.
Actual:	Row numbers 2, 3, 4 preserved accurately; citation links in UI click through to exact original row.
Justification:	*Guarantees 100% auditability and compliance: every hypothesis citation links directly to a verifiable line.*

TC-SCH-003 | Filename Sanitization Against Path Manipulation [POSITIVE] PASS

Test Input:	`Filename: '..\..\logs\incident:production.csv'`
Expected:	Sanitized to 'incident_production.csv'; path components stripped.
Actual:	sanitize_filename() returned 'incident_production.csv'; clean name used in DB and UI displays.
Justification:	*Prevents malformed filenames from corrupting HTTP response headers or database lookup keys.*

TC-SCH-004 | Multi-Format Timestamp Harmonization to UTC ISO [FUNCTIONAL] PASS

Test Input:	`Timestamps: '2026-10-28 14:15:00', '1730124900' (Unix epoch), '2026-10-28T14:15:00.000Z'`
Expected:	All formats normalized to standardized ISO-8601 string: '2026-10-28T14:15:00+00:00'.
Actual:	parse_iso_timestamp() converted all 3 formats to identical UTC ISO timestamps without timezone drift.
Justification:	*Accurate cross-service correlation requires a unified timeline regardless of origin server clock formats.*

TC-SCH-005 | Corrupt / Non-UTF8 Text Encoding Fallback [VALIDATION] PASS

Test Input:	`Upload CSV file containing corrupted high-ASCII byte sequences (e.g. ??)`
Expected:	Parser decodes with errors='replace', substituting ? without throwing fatal UnicodeDecodeError.
Actual:	File parsed successfully; invalid characters replaced cleanly; remaining text extracted intact.
Justification:	*Prevents investigation pipeline failure when processing garbled log segments from crashing hardware.*

MODULE 5: PRE-FLIGHT VALIDATION & TELEMETRY COVERAGE

Tests Step 5 data health checks, service discovery, gap analysis, and missing data warnings.

TC-VAL-001 | Pre-Flight Validation Metrics Calculation [POSITIVE] PASS

Test Input:	`GET /api/incidents/{id}/validation after uploading logs.csv (4 rows) and metrics.json (5 rows)`
Expected:	total_files=2, total_records=9, services_discovered=['api-gateway', 'checkout-service', 'database'], can_analyze=True.
Actual:	Validation summary calculated accurately; Step 5 summary metric cards displayed values correctly.
Justification:	*Gives investigators instant confirmation of ingested dataset scope prior to running the RCA algorithms.*

TC-VAL-002 | Missing Observability Telemetry Category Warnings [FUNCTIONAL] PASS

Test Input:	`Upload incident evidence containing only logs.csv (missing traces and metrics)`
Expected:	missing_categories=['traces', 'metrics', 'deployments', 'dependencies'], limitation_notice displayed.
Actual:	Notice displayed: 'Missing evidence categories: traces, metrics, deployments... Analysis can proceed with partial evidence.'
Justification:	*Ensures engineers understand telemetry blind spots and do not over-rely on partial diagnostic signals.*

TC-VAL-003 | Prevent RCA Trigger on Empty Evidence Dataset [VALIDATION] PASS

Test Input:	`POST /api/incidents/{id}/analyze on an incident with 0 uploaded records`
Expected:	HTTP 400 Bad Request with detail: 'Cannot analyze incident without evidence. Please upload logs, metrics, or traces first.'
Actual:	HTTP 400 returned; UI disabled analyze trigger button; empty analysis prevented.
Justification:	*Guards engine from executing mathematical correlation routines over null matrices.*

MODULE 6: EXPLAINABLE ROOT CAUSE ANALYSIS ENGINE

Tests heuristic ranking, contradiction detection, formula breakdowns, and engine abstention safeguards.

TC-RCA-001 | Deployment Regression Root Cause Identification [POSITIVE] PASS

Test Input:	`Scenario A: checkout-service v2.4.1 deployment followed by 504 timeouts and database lock errors`
Expected:	Rank 1 hypothesis: 'Recent deployment of checkout-service (v2.4.1) introduced a regression' (Score > 0.70).
Actual:	Rank 1 assigned with score 0.71; exact citations linked to deployments.csv row 2 and logs.csv row 3.
Justification:	*Solves the primary project theme: identifying that an outage was caused by a specific code change.*

TC-RCA-002 | Database Connection Pool Saturation & Deadlock Ranking [POSITIVE] PASS

Test Input:	`Scenario 2: billing-service HikariCP connection pool exhausted, active connections=100/100, lock deadlock`
Expected:	Rank 1: 'Database connection pool exhaustion and client acquisition timeout' with high confidence tier.
Actual:	Rank 1 assigned with score 0.74; supporting citations included connection saturation metrics and error logs.
Justification:	*Validates ability to diagnose backend database bottlenecks and resource exhaustion failures.*

TC-RCA-003 | Downstream 3rd-Party Latency Cascade Discovery [POSITIVE] PASS

Test Input:	`Scenario 3: stripe-external-api 10s latency causing payment-processor circuit breaker and edge 504s`
Expected:	Rank 1: 'Downstream dependency latency degradation causing upstream timeout cascade'.
Actual:	Rank 1 attributed correctly; traces mapped client duration of 10,250ms directly to partner API.
Justification:	*Isolates external vendor outages from internal bugs, preventing engineers from troubleshooting the wrong systems.*

TC-RCA-004 | Memory Saturation & Kernel OOM-Killer Attribution [POSITIVE] PASS

Test Input:	`Scenario 4: search-indexer memory climbing to 98.6%, GC pause=5400ms, OOM killer event logged`
Expected:	Rank 1: 'System resource saturation (CPU/Memory exhaustion) causing throttling and request drops'.
Actual:	Rank 1 attributed with score 0.71; memory metrics and kernel dmesg citations highlighted in analysis view.
Justification:	*Pinpoints memory leaks and process termination events before on-call engineers initiate blind restarts.*

TC-RCA-005 | Engine Abstention on Sparse Telemetry (Scenario B) [SAFEGUARD] PASS

Test Input:	`Scenario B: upload healthy baseline logs and metrics (HTTP 200, CPU=14%, 0 error signals)`
Expected:	Engine explicitly abstains (Score=0.10); outputs 'Insufficient evidence to rank a root cause reliably' with checklist.
Actual:	is_abstention=True returned; abstention card displayed in UI with recommended missing telemetry checklist.
Justification:	*Critical AI safety feature: platform refuses to invent false certainty when telemetry is inadequate.*

TC-RCA-006 | Contradiction & Falsification Detection [POSITIVE] PASS

Test Input:	`Scenario A dataset where database CPU was low (15%) while connection pool errors occurred`
Expected:	Engine cites database CPU as contradicting evidence, clarifying exhaustion was client-bounded.
Actual:	Contradicting evidence section populated: 'Database CPU remained low (15.0%), suggesting client-pool bounding.'
Justification:	*Ensures engineers review counter-signals and falsify incorrect assumptions during post-mortems.*

TC-RCA-007 | Bounded Heuristic Scoring Formula (0.05 to 0.96) [BOUNDARY] PASS

Test Input:	`Evaluate compute_heuristic_score() with extreme maximum and minimum input weights`
Expected:	Calculated score strictly clamped between 0.05 and 0.96; formula breakdown string generated.
Actual:	Extreme high inputs returned score 0.96; extreme low inputs returned score 0.05; clamp verified.
Justification:	*Mathematical boundary guarantee prevents ungrounded 0% or 100% certainty over probabilistic telemetry.*

MODULE 7: VISUALIZATION, EXPORT & MULTI-TENANCY

Tests UI visualizations (timeline, topology, metrics), audit report exports, and cross-user data isolation.

TC-UI-001 | Chronological Incident Timeline Rendering [POSITIVE] PASS

Test Input:	`Navigate to 'Incident Timeline' tab after running analysis on Scenario A`
Expected:	Interactive event sequence rendered in strict chronological order with severity color tags (Deployment, Error, Timeout).
Actual:	Timeline events rendered chronologically; timestamps ordered correctly; click opens evidence provenance modal.
Justification:	*Visual chronological reconstruction helps on-call teams comprehend the exact sequence of cascading failures.*

TC-UI-002 | **Service Dependency Topology Graph Rendering** [POSITIVE] PASS

Test Input:	`Navigate to 'Service Topology' tab after uploading service_dependencies.json`
Expected:	Directional graph renders nodes with computed health status (failing nodes highlighted in red).
Actual:	Graph rendered correctly; checkout-service node flagged as degraded; upstream/downstream links mapped.
Justification:	*Visualizes blast radius to indicate which customer-facing services are impacted by downstream failures.*

TC-EXP-001 | **Post-Incident Audit Report Generation (JSON & CSV)** [POSITIVE] PASS

Test Input:	`Click 'Export JSON' and 'Export CSV' buttons on Investigation Result page`
Expected:	Browser initiates download of audit-grade JSON file and formatted CSV table containing full citations and scoring.
Actual:	Files downloaded with Content-Disposition headers; JSON verified against schema; CSV formatted with RFC 4180.
Justification:	*Provides SRE teams with portable post-mortem artifacts for Jira, Confluence, and compliance audits.*

TC-MGMT-001 | **Multi-Tenant Data Isolation (Cross-User Incident Access Block)** [SECURITY] PASS

Test Input:	`User 2 attempts to fetch, update, or delete an incident owned by User 1 (GET /api/incidents/{user1_id})`
Expected:	HTTP 403 Forbidden with detail 'Access denied. You do not own this incident.'
Actual:	HTTP 403 Forbidden returned; database lookup verified user_id matching; cross-tenant leak prevented.
Justification:	*Enforces strict organizational multi-tenancy and data privacy across distinct engineering teams.*